

Rehearsal Script — ICAIBD Talk

Modality-Driven Search with Holistic Trace Judging for ARC-AGI-2

Johan Land

ICAIBD 2026

How to use this script. Each section is one slide. The [SAY] blocks are what you actually deliver — read them out loud a few times during rehearsal until they’re paraphraseable. The [CUE] blocks tell you when to advance the slide or point at the screen. Bracketed times are cumulative — glance at them if you suspect you’re drifting.

Slide 1 — Title (0:00 → 0:30)

[CUE] Title slide up. Stand still. Take one breath.

[SAY] > “Good morning. I’m Johan Land, and the title of the talk is *Modality-Driven Search with Holistic Trace Judging for ARC-AGI-2*. What I want to convince you of in the next fifteen minutes is this: on hard reasoning tasks, *how you search across hypotheses* matters more than which model produces them. I’ll show you a benchmark, the architecture, and the results — and then a few negative results that turned out to be the most important things I learned.”

[CUE] Advance to slide 2.

Slide 2 — ARC-AGI-2: a brief history + two examples (0:30 → 1:50)

[CUE] Two example problems on the left, “History of ARC-AGI” bullets on the right. Briefly point at each example, then walk through the bullets top-to-bottom with the laser pointer.

[SAY] > “Two ARC-AGI-2 examples on the left to give you a feel. *[Point at Example 1.]* Color the gray shapes using the legend at the top. *[Point at Example 2.]* Repair or complete the objects. The question marks are what the model has to predict — that’s the held-out test row. > > And a quick bit of history on the right, because it explains *why* the benchmark looks the way it does. > > *[Point at the first bullet.]* ARC was introduced in 2019 by François Chollet — framing intelligence as **skill-acquisition efficiency** rather than raw task performance. *[Next bullet.]* The first Kaggle competition in 2020 was won by DSL and program-synthesis approaches; LLMs scored under 5%. > > *[Next bullet.]* Then in late 2024, OpenAI’s o3 reportedly hit ~87% on the original ARC-AGI-1 — but at thousands of dollars per task. The benchmark was effectively saturated by brute-force test-time compute. > > *[Next bullet.]* So ARC-AGI-2 was launched in 2025, deliberately redesigned to make that strategy unaffordable: harder tasks, pass-at-two scoring, and **cost as a first-class metric**. Top frontier models dropped back to around 50%. *[Last bullet.]* And the

verified leaderboard sat in the 50s for months — **until this work, which is the first verified result above 70%, at 72.9%.**

[CUE] Advance to slide 3.

Slide 3 — What is ARC-AGI-2? — format and evaluation (1:50 → 2:50)

[CUE] JSON snippets on the left, three evaluation bullets on the right. Don't read the JSON line by line — just gesture at it.

[SAY] > “A quick look at how this actually works. *[Gesture at the left column.]* The model gets the task as JSON: a list of training input/output pairs, plus one or more test inputs whose outputs are hidden. It returns its prediction as a grid — just a 2D array of integers. > > Three things to flag on the right. > > **Pass-at-two:** the model is allowed two guesses per test input. If either matches the ground truth exactly, it counts. This matters later — it's why the judging architecture I'll show outputs two hedged predictions, not one. > > **The official scoring is private.** A public eval split is provided for development, but the score that goes on the leaderboard comes from the ARC Prize foundation running the solver end-to-end on a private eval split that the developer never sees. That's what makes the benchmark hard to game. > > **And cost is graded alongside accuracy** — dollars per task is a first-class metric, not a footnote.”

[CUE] Advance to slide 4.

Slide 4 — Headline result (2:50 → 3:35)

[CUE] Leaderboard table. Don't read every row — point at your row, then GPT-5.2 Pro's row.

[SAY] > “This is the headline result. The system I'll describe gets **72.9%** on the official ARC Prize Verified leaderboard, at about \$39 per task. On the public evaluation it's 76%, at \$19 per task. *[Point at GPT-5.2 Pro row.]* The strongest standalone frontier model is here at 54.2%. That's +18.7 percentage points. > > Yes — this spends more than a single model call. The contribution I'm presenting today is the **architectural pattern**, not the specific leaderboard number, which will move within weeks. The question is: *how* does an orchestration system beat the best single model by 18 points?”

[CUE] Advance to slide 5.

Slide 5 — Selection, not generation (3:35 → 5:05)

[CUE] Bullet slide. Slow down here — this is the reframe.

[SAY] > “Here's the reframe that unlocks everything else. > > When LLMs reason about hard ARC tasks, they produce fluent, internally coherent reasoning traces — and they're often *confidently wrong*. Worse: on the hardest tasks, multiple models tend to **cluster around the same wrong interpretation**. They all see the same surface pattern and miss the same latent rule. > > Which means majority voting or self-consistency makes things *worse* — it amplifies the wrong answer. So the hard problem here isn't generating *a* correct candidate. It's **recognizing** one when most of

your candidates disagree with it. > > To put a number on it: in my failure analysis, out of 39 failed instances, only 21 are genuine generation failures — meaning no correct candidate exists in the pool. The other 17 are *selection* failures: a correct candidate **does** exist, but the system didn't pick it. That's the problem this architecture is trying to solve."

[CUE] Advance to slide 6.

Slide 6 — Modalities as search operators (5:05 → 6:35)

[CUE] Three-column slide: TEXT / IMAGE / CODE. This is the slide they should remember.

[SAY] > "So if the correct answer is often a minority hypothesis, we need two things: a way to *generate* that minority hypothesis in the first place, and a way to *recognize* it when most candidates disagree. > > The architectural insight on the generation side is this: **reasoning modalities are search operators**. Take the same ARC task and present it to a model three different ways. [*Point at columns.*] Have it read the grid as text — numbers and strings. Have it look at a rendered PNG of the grid. Or have it write Python code that transforms the grid. Same task, three representations — three completely different hypothesis distributions. > > The intuition is concrete: a model reading grids as numbers might notice an arithmetic pattern that the vision-based pass completely misses. A code-writing pass forces the model to commit to a *constructive* rule that the other modalities can leave implicit. > > So I generate candidates **independently** across these three families. I deliberately don't pool prompts or share scaffolding between them — that would defeat the point. The diversity I want is at the *representation* level, not at the temperature level."

[CUE] Advance to slide 7.

Slide 7 — Pipeline overview (6:35 → 7:35)

[CUE] Pipeline diagram. One sweep across, no more than 50 seconds.

[SAY] > "Here's the end-to-end pipeline. Three steps. > > [*Sweep left to right.*] First, **generate**: up to 29 candidates per task, distributed across the three modality families, using three frontier models — GPT-5.2, Gemini 3 Preview, and Claude Opus 4.5. Each candidate produces a predicted output grid *and* its full reasoning trace. > > Second, **judge holistically**: three judge models, in parallel, each one reads *every* candidate's full trace in one long-context prompt. They each pick a top-two — or propose a synthesis that recombines pieces of multiple candidates. > > Third, **weighted vote**: first-place pick gets 2 points, second gets 1. Highest-scoring two grids become the pass@2 guesses. > > One efficiency note: there's an adaptive early-stop. If the first 8-candidate probe already shows strong agreement, we skip the expensive modalities. On the public eval, 37 of 167 instances stopped early — and 36 of those 37 were correct. So 97% precision on the early-exit path."

[CUE] Advance to slide 8.

Slide 8 — Candidate generation in practice (7:35 → 8:35)

[CUE] Bullet slide. Brisk pace.

[SAY] > “A bit more concrete on the generation phase. Three frontier models. Three modality families. Twenty-nine candidates per task, in total, across configurations within each family. Code candidates get access to a Python sandbox so they can iterate — write code, run it on the training pairs, see if the output matches, debug. > > One bullet I want to flag — **deliberately minimal prompts**. I’ll come back to why on the negative-results slide.”

[CUE] Advance to slide 9.

Slide 9 — Holistic judging vs. majority voting (8:35 → 10:05)

[CUE] +7 number is the centerpiece. Say it slowly.

[SAY] > “Now the selection side. Once we have 29 candidates per task, the question is how to pick. > > The architecture is this: all 29 reasoning traces — every one, in full — go into a single long-context prompt for the judge. The judge identifies the top two candidates and, importantly, **explains why the other clusters are wrong**. I run three judges in parallel and weight-vote across them. > > The empirical payoff: compared to a simple majority-vote baseline — pick the most common candidate output — this judging architecture recovers **+7 additional solved instances**. And all seven are minority recoveries: cases where the correct answer was *not* the most common candidate. The judge could only find them by reading the *reasoning*, not by counting votes. > > There’s one more case — **synthesis**. One instance, 21897d95:2, was solved even though *none* of the 29 candidates were correct. The judge took partial insights from several wrong candidates and recombined them into a novel correct output. One instance is a small number, but it’s qualitatively different — it shows the architecture can sometimes *construct* a correct answer, not just *select* one. Compositional repair, in principle.”

[CUE] Advance to slide 10.

Slide 10 — End-to-end walkthrough (10:05 → 12:35)

[CUE] This is the demo slide. Budget the full 2:30. Don’t rush.

[SAY] > “Let me make this concrete with one task. *[Bring up the task image.]* This is task d35bdbdc. Take a second to look at it. *[Pause 5 seconds.]* > > When I run the Text candidates on this, the dominant interpretation looks like *[describe what the text models converge on — adapt to whichever side of the figure you’re pointing at]*. Most Text candidates pick that interpretation. It’s plausible. It even works on most of the training pairs. > > The Code candidates — because they have to write executable Python — end up forcing a slightly different interpretation. *[Point at code-cluster outputs.]* Notice the structural difference: this is the same task, but the *representational pressure* is different. > > The Image candidates split between the two. *[Point.]* > > Now the judge reads all of these traces together. *[Point at judge decision.]* It reasons about the disagreement between the clusters explicitly — the rationale we logged for this kind of case literally says things like ‘some solvers assume a stamp must fit fully; others assume stamps are clipped at the border.’ And then it uses the pass@2 format to **hedge** across the two interpretations — one guess for each. > > That’s what *modalities as search operators* looks like in practice.”

[CUE] Advance to slide 11.

Slide 11 — Modality complementarity (12:35 → 13:35)

[CUE] Methodology matrix figure. Point at the table numbers.

[SAY] > “Some quantitative evidence that the modalities really are doing different work. > > [*Point at numbers.*] Restricting to the 130 instances with full candidate coverage: Code solves 18 instances that Text doesn’t. Image solves 13 that Text doesn’t. At the *exclusive* level — instances solved by **only one** family — we see 2 Text-only, 6 Image-only, 7 Code-only. So they’re not redundant copies of the same reasoning process. > > One number to anchor on: **86.2%**. That’s the candidate-oracle accuracy — the fraction of instances where at least one of the 29 candidates is correct. We solve 76.6%, so we’re leaving about 10 percentage points on the table at the *selection* step. That’s where the next round of work should go.”

[CUE] Advance to slide 12.

Slide 12 — Negative results (13:35 → 14:35)

[CUE] Two-claim slide. Slow down for both punchlines.

[SAY] > “I want to end on the two most counter-intuitive findings, because they shaped every other decision in the system. > > First: **prescriptive prompting degrades performance**. Reasoning templates, chain-of-thought scaffolding, domain-specific heuristics — every time I added ‘helpful’ structure to a prompt, scores on the hardest tasks went *down*. The mechanism appears to be a **compliance tax on reasoning**: when you tell a model exactly *how* to think, it spends test-time compute following the template instead of exploring unconventional hypotheses. > > Second: **iterative refinement reduces diversity**. If you refine candidates against each other — let them see each other’s outputs and try to improve — they collapse onto the same wrong cluster. Which is the exact failure mode we’re trying to escape. > > So the final system uses **deliberately minimal prompts** and **fully independent generation**. The architecture earns its diversity through representational variety, not through prompt engineering.”

[CUE] Advance to slide 13.

Slide 13 — Takeaways (14:35 → 15:05)

[CUE] Three-bullet recap. Don’t add new material.

[SAY] > “Three things to take home. > > One — **modalities are search operators**. Text, image, and code activate different hypothesis distributions, and you can exploit that. > > Two — **holistic judging beats majority voting** on hard tasks, because the correct answer is often a minority hypothesis and you need to read the reasoning, not count votes. > > Three — **less prompt structure, more representational diversity**. The architecture wins when the lanes are independent and the prompts get out of the way. > > Code is open source — link’s on the slide. Happy to take questions.”

[CUE] Stop. Look up. Wait for questions.

Q&A — likely questions and short answers

Q. Isn't \$39/task expensive? > “Yes. The representative number is actually the public-eval cost of \$19/task — the semi-private cost was inflated by a known GPT-5.2 API instability period that gave us an 84% API call failure rate. At \$19/task the system is competitive with GPT-5.2 Pro on cost and 22 points higher on accuracy. There's also an adaptive early-stop that makes easy tasks much cheaper — on ARC-AGI-1 the same system runs at \$11/task.”

Q. Why three judges instead of one? > “A single judge is biased toward its own prior. Three parallel judges with weighted voting — 2 points first pick, 1 point second pick — hedges within the judging step itself. And pass@2 hedges again at the output level when judges disagree.”

Q. What's the dominant failure mode now? > “Selection, by a noticeable margin. 17 of 39 failures are cases where a correct candidate exists in the candidate pool but the judge didn't pick it. Generation is in better shape than selection.”

Q. Does this generalize beyond ARC? > “I think the *pattern* generalizes — multi-modal generation plus holistic long-context judging — anywhere you have a verifiable task with multiple plausible hypotheses and where minority hypotheses are systematically correct. ARC happens to expose this cleanly because tasks are short and pass@2 is generous, but I'd expect the architecture to transfer to, say, mathematical reasoning or program synthesis tasks with similar structure.”

Q. Why did you pick those three models specifically? > “Empirical: I tested several combinations and these three contributed the most *unique* solves — meaning each model solved at least some instances no other model solved. The exact model mix will rotate as new frontier models come out; the architectural pattern is what's stable.”

Q. Did you solve the two example tasks at the start? > “Both are in the public evaluation split, and yes — the final system solves both. They were chosen as representative of the *kinds* of abstractions ARC-AGI-2 contains, not as failure cases.”

Backup slide cues (only used if asked)

- **Cost details** — backup slide 14
- **Failure decomposition** — backup slide 15
- **Why three judges** — backup slide 16

Self-check before going on stage

- ☐ Slide 2 (history + examples) stays under 1:20 — the timeline can swallow time if you elaborate on each beat
- ☐ Slide 3 (What is ARC) stays under 1:00 — the historical context is already on slide 2, don't repeat it
- ☐ Slide 10 demo stays under 2:30 (second most likely to overrun)
- ☐ You can quote the +7 minority-recovery number from memory
- ☐ You can quote the 86.2% oracle-ceiling number from memory

- ☐ You know which row on the leaderboard table you're pointing at
- ☐ Backup slides 14–16 are loaded and you remember the slide numbers